For most of these, the `nltk` and `scikit-learn` libraries will be used. For Word Embeddings, you'll need `gensim`.

- **Install them:**

```
pip install nltk scikit-learn gensim
```

- **NLTK Downloads:** `nltk` also requires you to download data packages. You only need to do this **once**. I'll include the download commands in the code where needed.

---

1. Lowercasing

This is the easiest step. It's just a built-in Python string method.

```
text = "Hamna is learning NLP! She loves Python."
lower_text = text.lower()

print(f"Original: {text}")
print(f"Lowercase: {lower_text}")
```

Output:

```
Original: Hamna is learning NLP! She loves Python.
Lowercase: hamna is learning nlp! she loves python.
```

---

2. Remove Punctuation and Numbers

The easiest way is to use Python's `string` library and `str.maketrans`.

```
import string

text = "hamna loves python 123! it's great."

# string.punctuation is '!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
# string.digits is '0123456789'
to_remove = string.punctuation + string.digits
translation_table = str.maketrans('', '', to_remove)

clean_text = text.translate(translation_table)

print(f"Original: {text}")
print(f"Cleaned: {clean_text}")
```

Output:

```
Original: hamna loves python 123! it's great.
Cleaned: hamna loves python  its great
```

---

3. Tokenization

This splits a sentence into a list of words (tokens).

```
import nltk
from nltk.tokenize import word_tokenize

# You only need to run this download once
# nltk.download('punkt')

text = "Hamna loves learning Python."
tokens = word_tokenize(text)

print(f"Sentence: {text}")
print(f"Tokens: {tokens}")
```

Output:

```
Sentence: Hamna loves learning Python.
Tokens: ['Hamna', 'loves', 'learning', 'Python', '.']
```

---

4. Stop Word Removal

This removes common words (like 'is', 'a', 'the') that don't add much meaning.

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# You only need to run this download once
# nltk.download('stopwords')
# nltk.download('punkt')

text = "Hamna is learning a lot about Python and she loves it."
tokens = word_tokenize(text.lower())

# Get the set of English stop words
stop_words = set(stopwords.words('english'))

filtered_tokens = []
for word in tokens:
    if word not in stop_words:
        filtered_tokens.append(word)

print(f"Original Tokens: {tokens}")
print(f"Filtered Tokens: {filtered_tokens}")
```

Output:

```
Original Tokens: ['hamna', 'is', 'learning', 'a', 'lot', 'about', 'python', 'and', 'she', 'loves', 'it', '.']
Filtered Tokens: ['hamna', 'learning', 'lot', 'python', 'loves', '.']
```

---

5. Stemming

Reduces words to their root form. It's a crude, fast method that just chops off the end.

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = ["learning", "models", "working", "studies"]

stemmed_words = [stemmer.stem(w) for w in words]

print(f"Original: {words}")
print(f"Stemmed: {stemmed_words}")
```

Output:

```
Original: ['learning', 'models', 'working', 'studies']
Stemmed: ['learn', 'model', 'work', 'studi']
```

---

6. Lemmatization

Reduces words to their dictionary form (lemma). It's smarter than stemming but slower.

```
import nltk
from nltk.stem import WordNetLemmatizer

# You only need to run this download once
# nltk.download('wordnet')
```

```
lemmatizer = WordNetLemmatizer()
words = ["models", "wolves", "better", "is"]

# We use pos='v' for verbs, pos='a' for adjectives. Default is 'n' (noun).
lemmatized_words = [
    lemmatizer.lemmatize("models"),        # Noun
    lemmatizer.lemmatize("wolves"),        # Noun
    lemmatizer.lemmatize("better", pos='a'), # Adjective
    lemmatizer.lemmatize("is", pos='v')      # Verb
]

print(f"Original: {words}")
print(f"Lemmatized: {lemmatized_words}")
```

Output:

```
Original: ['models', 'wolves', 'better', 'is']
Lemmatized: ['model', 'wolf', 'good', 'be']
```

---

7. POS Tagging (Part-of-Speech)

Tags each word with its part of speech (e.g., Noun, Verb, Adjective).

```
import nltk
from nltk.tokenize import word_tokenize

# You only need to run these downloads once
# nltk.download('punkt')
# nltk.download('averaged_perceptron_tagger')

text = "Hamna is learning Python quickly."
tokens = word_tokenize(text)
pos_tags = nltk.pos_tag(tokens)

print(pos_tags)
```

Output:

```
[('Hamna', 'NNP'), ('is', 'VBZ'), ('learning', 'VBG'), ('Python', 'NNP'), ('quickly', 'RB'), ('.', '.')]
```

(NNP = Proper Noun, VBZ = Verb, VBG = Verb, RB = Adverb)

---

8. Bag of Words (BoW)

Represents text as a count of its words, ignoring order.

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "Hamna loves Python",
    "Python is great",
    "Hamna also loves NLP"
]
vectorizer = CountVectorizer()
bow_matrix = vectorizer.fit_transform(corpus)

print(f"Vocabulary: {vectorizer.get_feature_names_out()}")
print("BoW Matrix:")
print(bow_matrix.toarray())
```

Output:

```
Vocabulary: ['also' 'great' 'hamna' 'is' 'loves' 'nlp' 'python']
BoW Matrix:
[[0 0 1 0 1 0 1]
```

```
[0 1 0 1 0 0 1]
[1 0 1 0 1 1 0]]
```

- **Row 1:** 'Hamna' (1), 'loves' (1), 'Python' (1)
- **Row 2:** 'great' (1), 'is' (1), 'Python' (1)
- **Row 3:** 'also' (1), 'Hamna' (1), 'loves' (1), 'NLP' (1)

---

9. TF-IDF (Term Frequency-Inverse Document Frequency)

Like BoW, but gives more weight to words that are important (frequent in one document, but rare in all documents).

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "Hamna loves Python",
    "Python is great",
    "Hamna also loves NLP"
]
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(corpus)

print(f"Vocabulary: {vectorizer.get_feature_names_out()}")
print("TF-IDF Matrix (rounded to 2 decimal places):")
print(tfidf_matrix.toarray().round(2))
```

Output:

```
Vocabulary: ['also' 'great' 'hamna' 'is' 'loves' 'nlp' 'python']
TF-IDF Matrix (rounded to 2 decimal places):
[[0.   0.   0.55 0.   0.55 0.   0.63]
 [0.   0.7  0.   0.7  0.   0.   0.5 ]
 [0.63 0.   0.43 0.   0.43 0.63 0.  ]]
```

---

10. Word Embedding (using `gensim` Word2Vec)

This is the most complex concept. This code trains a *tiny* model that learns vector representations for words. Words with similar meanings will have similar vectors.

```
from gensim.models import Word2Vec

# Your corpus must be a list of lists of tokens
sentences = [
    ['hamna', 'loves', 'python'],
    ['hamna', 'loves', 'nlp'],
    ['python', 'is', 'a', 'great', 'language']
]

# Train a very simple model
# vector_size=5: Each word gets a 5-dimensional vector
# window=2: Looks at 2 words before and 2 words after
# min_count=1: Counts all words, even if they only appear once
model = Word2Vec(sentences, vector_size=5, window=2, min_count=1)

# Get the learned vector for a word
vector_python = model.wv['python']

# Find words similar to 'hamna'
similar_to_hamna = model.wv.most_similar('hamna', topn=1)

print(f"Vector for 'python': \n{vector_python.round(2)}")
print(f"Most similar to 'hamna': {similar_to_hamna}")
```

Output (will vary slightly each time you run it):

```
Vector for 'python':
[-0.03  0.08  0.02 -0.05  0.07]
```

```
Most similar to 'hamna': [('python', 0.1656...)]
```

(The similarity score is low because the dataset is extremely small, but it shows the code works!)